transforming Pipeline Validation
Service into a general purpose GitLab Policy Service / GitLab Policy Agent that will be well integrated into the GitLab product itself.

Generalizing Pipeline Validation Service into GitLab Policy Service can bring a few interesting benefits:

1. Consolidate on our tooling across the company to improve efficiency.
1. Integrate our GitLab Rails limits framework to resolve policies using the policy service.
1. Do not struggle to define complex policies in YAML and hack evaluating them in Ruby.
1. Build a policy for GraphQL queries limiting using query execution cost estimation.
1. Make it easier to resolve policies that do not need "hierarchical limits" structure.
1. Make GitLab Policy Service part of the product and integrate it into the single application.

We envision using GitLab Policy Service to be place to define policies that do not require knowing anything about the hierarchical structure of the limits. There are limits that do not need this, like IP addresses allow-list, spam checks, configuration validation etc.

We defined "Policy" as a stateless, functional-style, limit. It takes input arguments and evaluates to either true or false. It should not require a global counter or any other volatile global state to get evaluated. It may still require to have a globally defined rules / configuration, but this state is not volatile in a same way a rate limiting counter may be, or a megabytes consumed to evaluate quota limit.

Policies used internally and externally

The GitLab Policy Service might be used in two different ways:

1. Rails limits framework will use it as a source of policies enforced internally.
1. The policy service feature will be used as a backend to store policies defined by users.

These are two slightly different use-cases: first one is about using internally-defined policies to ensure the stability / availability of a GitLab instance (GitLab.com or self-managed instance). The second use-case is about making GitLab Policy Service a feature that users will be able to build on top of.

Both use-cases are valid but we will need to make technical decision about how to separate them. Even if we decide to implement them both in a single service, we will need to draw a strong boundary between the two.

The same principle might apply to Decouple Limits Service described in one of the sections of this document above.

The two limits / policy services

It is possible that GitLab Policy Service and Decoupled Limits Service can actually be the same thing. It, however, depends on the implementation details

that we can't predict yet, and the decision about merging these services together will need to be informed by subsequent iterations' feedback.

Hierarchical limits

GitLab application aggregates users, projects, groups and namespaces in a hierarchical way. This hierarchical structure has been designed to make it easier to manage permissions, streamline workflows, and allow users and customers to store related projects, repositories, and other artifacts, together.

It is important to design the new rate limiting framework in a way that it built on top of this hierarchical structure and engineers, customers, SREs and other stakeholders can understand how limits are being applied, enforced and overridden within the hierarchy of namespaces, groups and projects.

We want to reduce the cognitive load required to understand how limits are being managed within the existing permissions structure. We might need to build a simple and easy-to-understand formula for how our application decides which limits and thresholds to apply for a given request and a given actor:

> GitLab will read default limits for every operation, all overrides configured
> and will choose a limit with the highest precedence configured. A limit
> precedence needs to be explicitly configured for every override, a default
> limit has precedence 100.

One way in which we can simplify limits management in general is to:

1. Have default limits / thresholds defined in YAML files with a default precedence 100.
1. Allow limits to be overridden through the API, store overrides in the database.
1. Every limit / threshold override needs to have an integer precedence value provided.
1. Build an API that will take an actor and expose limits applicable for it.
1. Build a dashboard showing actors with non-standard limits / overrides.
1. Build a observability around this showing in Kibana when non-standard limits are being used.

The points above represent an idea to use precedence score (or Z-Index for limits), but there may be better solutions, like just defining a direction of overrides - a lower limit might always override a limit defined higher in the hierarchy. Choosing a proper solution will require a thoughtful research.

Principles

1. Try to avoid building rate limiting framework in a tightly coupled way.
1. Build application limits API in a way that it can be easily extracted to a separate service.
1. Build application limits definition in a way that is independent from the Rails application.
1. Build tooling that produce consistent behavior and results across programming languages.
1. Build the new framework in a way that we can extend to allow self-managed administrators to customize limits.
1. Maintain consistent features and behavior across SaaS and self-managed codebase.
1. Be mindful about a cognitive load added by the hierarchical limits, aim to reduce it.

Phases and iterations

1. Compile examples of current most important application limits (Owning Team)
   - Owning Team (in collaboration with Stage Groups) compiles a list of the most important application limits used in Rails today.

1. Implement Rate Limiting Framework in Rails (Owning Team)
   - Triangulate rate limiting abstractions based on the data gathered in Phase 1.
   - Develop YAML model for limits.
   - Build Rails SDK.
   - Create examples showcasing usage of the new rate limits SDK.

1. Team fan out of Rails SDK (Stage Groups)
   - Individual stage groups begin using the SDK built in Phase 2 for new limit and policies.
   - Stage groups begin replacing historical ad hoc limit implementations with the SDK.
   - (Owning team) Provides means to monitor and observe the progress of the replacement effort. Ideally this is broken down to the featurecategory level to drive group-level buy-in.

1. Enable Satellite Services to Use the Rate Limiting Framework (Owning Team)
   - Determine if the goals of Phase 4 are best met by either:
     - Extracting the Rails rate limiting service into a decoupled service.
     - Implementing a separate Go library which uses the same backend (for example, Redis) for rate limiting.

1. SDK for Satellite Services (Owning Team)
   - Build Go SDK.
   - Create examples showcasing usage of the new rate limits SDK.

1. Team fan out for Satellite Services (Stage Groups)
   - Individual stage groups begin using the SDK built in Phase 5 for new limit and policies.
   - Stage groups begin replacing historical ad hoc limit implementations with the SDK.

Status

Request For Comments.

Timeline

- 2022-04-27: Rate Limit Architecture Working Group started.
- 2022-06-07: Working Group members started submitting technical proposals for the next rate limiting architecture.
- 2022-06-15: We started scoring proposals submitted by Working Group members.
- 2022-07-06: A fourth, consolidated proposal, has been submitted.
- 2022-07-12: Started working on the design document following Architecture Evolution Workflow.
- 2022-09-08: The initial version of the blueprint has been merged.

SECTION: engineering/architecture/design-documents/rate_limiting_simplification/_index.md

<!-- Design Documents often contain forward-looking statements -->
<!-- value gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

Summary

As GitLab.com has evolved, we've introduced rate limits and throttling measures at many layers to enhance security and performance of our platform, ensuring availability and user satisfaction are at a desired level. These measures have often been reactionary, and we have not had a consistent strategy for defining and enforcing these limits.

This lack of consistency can lead to confusion for our users and loss of productivity for our engineers, as well as difficulty when we wish to define and implement new limits. This Design Document aims to introduce a transparent strategy for these limits; firstly in order to enable our engineers to introduce sensible policies to enforce our availability goals, and secondly allow us transparency to expose these to our users.

With the introduction of Cells, we are taking a proactive and iterative approach to simplifying how we define and manage our rate limits at all layers going forward.

This document is intended to define iterative steps that support the existing Next Rate Limiting Architecture which focuses on Application limits, and builds on this to encompass edge network limits.

Motivation

Presently the Rate Limiting for GitLab.com is defined in several different places and in many different ways. This creates a number of challenges for our customers and team members alike.

Firstly, the spread of places rate limits exist can make it difficult to understand which limits have been applied to a request. This leads to confusion for our users and productivity loss for our support engineers and production engineering teams. Finding which limits have been applied can require investigation into several tools and codebase audits, since it is not always clear where a limit has been reached (for example in Cloudflare or the GitLab Application).

Secondly, it can be difficult to change or introduce new limits, and as such define policies for protecting our availability. As different limits are enforced in different places, there is no single way to change them for a namespace / project / user / customer. This has led to inconsistencies when new services are deployed, further adding to the confusion of which limits are applied to a request, or can mean services are deployed without any rate limiting at all. Introducing throttling after a service or an API is deployed can come with risks, as it can cause unexpected interruptions to customers.

Finally, we have historically allowed rate limit bypasses for some marquee SaaS customers, which increases risk for our platform and those customers while being difficult to manage long-term. We would like to be able to set higher limits to allow us to protect our availability while also providing user satisfaction to those customers.

As GitLab.com transitions to a Cells architecture, we have a unique opportunity to create a single source of truth for our rate limits. In the longer term we will work towards a centralised location for rate limits to be configured with sensible defaults, that also allows for customisation for a namespace / project / user / customer as needed. This will also provide a benefit of easily documentable and discoverable rate limits that we can publish to our users.

Current Rate Limiting Configuration Methods

This diagram is intended to illustrate the complexities in managing our rate limits today across each layer of our stack, and across each of our product offerings.

```
mermaid
flowchart LR
    subgraph configuration mechanism

        config-mgmt[Terraform - config-mgmt]
        waf-module[Terraform - cloudflare-waf-rules module]

        chef-repo
        haproxy-cookbook

        hardcoded
        API
        app-settings[Application Settings]
        k8s-helmfiles
        vault

        manual
    end

    subgraph GitLab.com
        com-cloudflare[Cloudflare]
        com-haproxy[HAProxy]
        com-app[Application]
    end

    subgraph Dedicated
        d-cloudflare[Cloudflare]
        d-app[Application]
    end

    subgraph Self-Managed
        sm-app[Application]
    end

    subgraph Cells
        cell-cloudflare[Cloudflare]
        cell-app[Application]
    end
```

```
%% ==================================
%% GitLab.com Configuration Mechanisms
%% ==================================

%% Cloudflare
config-mgmt --> com-cloudflare

%% HAProxy
chef-repo --> com-haproxy
haproxy-cookbook --> com-haproxy

%% Application
hardcoded --> com-app
API --> com-app
app-settings --> com-app
k8s-helmfiles --> com-app
vault --> com-app
vault --> k8s-helmfiles

%% Documentation
manual --> Documentation

%% ==================================
%% Self-Managed Configuration Mechanisms
%% ==================================

%% Application
hardcoded --> sm-app
API --> sm-app
app-settings --> sm-app

%% ==================================
%% Dedicated Configuration Mechanisms
%% ==================================

%% Cloudflare
waf-module --> d-cloudflare

%% Application
hardcoded --> d-app
API --> d-app
app-settings --> d-app

%% ==================================
%% Cells Configuration Mechanisms
%% ==================================

%% Cloudflare
waf-module --> cell-cloudflare
```

```
%% Application
hardcoded --> cell-app
API --> cell-app
app-settings --> cell-app
```

Technical Roadmap

This initiative focuses on enabling us to improve the performance, stability, and scalability of GitLab through making rate limiting configuration easier to manage, and reducing the cognitive overhead of needing to investigate what rates limits are configured, and where (as illustrated above). This has been seen as a common pain point across Engineering and Support, therefore focusing on iterative improvements we can make to this configuration will put us in a more secure position.

Additionally, the proposed architecture encourages the development of a single, strategic cross-application method for consistent rate-limiting, over multiple siloed and piecemeal implementations, which only add to complexity and cognitive overhead for GitLab team members and customers.

Goals

Simplify rate limiting across the GitLab ecosystem.

- Support the ability to configure rate limits in version control.
- Introduce a process for defining new rate limits.
- Introduce a set of "default" rate limits for new services, and enforce limits on deployment of new services in an automated way.
- Simplify tweaking of limits for production engineers (for instance during an incident).
- Create transparency for team members and our customers to understand where requests are being limited and why.

Goals for each phase

- Phase 1: Consolidate the infrastructure rate limit configuration for limits and allow-listing in one centralized place.
- Phase 2: Consolidate the application rate limit configuration in one centralized place.
- Phase 3: Provide a rate limiting interface to make managing configuration and publishing limits easier.

Non-Goals

- To re-implement Cloudflare or Rack Attack or a throttling service - this is meant to be an interface for consolidating and defining limits, as an abstraction not a throttler.

Proposal

Modify GitLab's existing rate limiting architecture to support passing in rate limiting configuration files, and once this is supported create an interface to simplify the configuration and management of these limits across environments.

Phase 1: Simplify our edge network and bypass configuration

- Phase 1.1: Manage IP-based rate limiting bypasses in one location
  - Migrate bypass header logic out of HAProxy and into Cloudflare.
  - Cloudflare custom rules support transform-rule actions which should make this possible.
- Phase 1.2: Support passing in a configuration file for Cloudflare rules
  - Migrate Cloudflare rules to use the cloudflare-waf-rules Terraform module.
  - Use terraform-vars to manage configuration of these rules.

Phase 2: Simplify application level configuration

At this point, this proposal refers to the framework mentioned in the Next Rate Limiting Architecture by introducing a mechanism to define limits in a structured way, making sure that our existing code-paths utilise that framework to enforce limits.

- Support passing in a configuration file for RackAttack throttles
  - Some are configurable, some are hard coded [source].
- Support passing in a configuration file for ApplicationRateLimiter throttles
  - Some are configurable through Application Settings UI, some through the API, some are hard coded [source].
- Support the ability to pass in configuration file for all application limits
  - There are seven different types of application limits (if you count RackAttack and ApplicationRateLimiter separately), we eventually want to be able to configure these through the same configuration mechanism.
  - See this note (confidential) for the full list of limiting implementations.
- Support configuring limits across services outside the GitLab-rails application
  - Some services are deployed with Runway, and should support a standardized rate limiting configuration mechanism.
    - See the issue around rate limiting Runway services here.
  - Further research will need to be done to account for services deployed outside of Runway.

mermaid
flowchart LR
    style com-haproxy stroke-dasharray: 5 5

    subgraph configuration mechanism
        cf-config[tfvars]
        app-config[YAML config file]

        waf-module[Terraform - cloudflare-waf-rules module]

        chef-repo
        haproxy-cookbook

        API
        app-settings[Application Settings]

        manual

```
end

subgraph GitLab.com
   com-cloudflare[Cloudflare]
   com-haproxy[HAProxy]
   com-app[Application]
end

subgraph Dedicated
   d-cloudflare[Cloudflare]
   d-app[Application]
end

subgraph Self-Managed
   sm-app[Application]
end

subgraph Cells
   cell-cloudflare[Cloudflare]
   cell-app[Application]
end

%% config files
cf-config --> waf-module
app-config --> com-app
app-config --> sm-app
app-config --> d-app
app-config --> cell-app

API --> com-app
API --> sm-app
API --> d-app
API --> cell-app

%% Cloudflare
waf-module --> com-cloudflare
waf-module --> d-cloudflare
waf-module --> cell-cloudflare

%% HAProxy
%% Still exists for Pages and Registry but isn't a priority
chef-repo --> com-haproxy
haproxy-cookbook --> com-haproxy

%% Application Settings
app-settings --> com-app
app-settings --> sm-app
app-settings --> d-app
app-settings --> cell-app
```

```
%% Documentation
manual --> Documentation
```

Phase 3: Implement a Rate Limit Interface

Create a Rate Limiting interface for configuration of rate limits across all parts of GitLab.com and Cells. While this interface will not be responsible for enforcing limits (throttling will still take place in Cloudflare, or the application itself, for example), it will provide a consolidated catalogue of limits, as well as providing a mechanism to override any limits per Namespace. To provide this single source of truth, any new throttles that get added need to be reflected in the catalogue with their default values. If these default-limits are implemented in the application, this means that the value in the catalogue is what packages for self-managed will ship with.

The Rate Limiting Single Source of Truth (SSOT) should focus on configuring rate limiting rules and thresholds in an abstracted manner regardless of the mechanism that is used to apply these rules to the underlying systems. For example, this might look like a well-defined YAML file and schema, defining Rate Limiting from GitLab·s domain-specific-view (customers, organizations, cells, features, etc), rather than directly as transport semantics (eg, endpoints and HTTPS). This definition can then be transformed into Cloudflare, GitLab Application (or even as-yet-undetermined future technologies) as configuration through the use of CI workflows and supporting scripts.

In practice, what this might look like:

1. A new rule is merged into to the Rate Limit Source of Truth, producing a new version.
1. An automation raises a corresponding MR to update the rate limiting configuration in the respective repositories.
1. Upon merge the new limits are applied using configuration that will apply to Cloudflare or loaded into the GitLab Application.
1. A CI workflow run will parse and update the rate limit thresholds in our documentation.

Note: the specifics about this implementation are subject to change, as we progress through the first two phases and learn more.

```
mermaid
flowchart LR
   style com-haproxy stroke-dasharray: 5 5
   style chef-repo stroke-dasharray: 5 5
   style haproxy-cookbook stroke-dasharray: 5 5

   subgraph configuration mechanism
      chef-repo
      haproxy-cookbook

      ssot[Rate Limiting SSOT]

      cf-config[tfvars]
      waf-module[Terraform - cloudflare-waf-rules module]
```

```
    subgraph app-config-mechanisms[Application configuration]
      app-config[YAML config file]
      API
      app-settings[Application Settings]
    end
  end

  subgraph platforms[All GitLab Platforms]
    com-cloudflare[Cloudflare]
    subgraph com-app[Application]
      com-rl-ss[Rate Limit Satellite Service]
    end
  end

  subgraph com[GitLab.com]
    com-haproxy[HAProxy]
  end
  %% single source of truth
  ssot -- CI workflow --> cf-config
  ssot -- CI workflow --> app-config

  %% config files
  cf-config --> waf-module
  app-config-mechanisms --> com-app

  %% Cloudflare
  waf-module --> com-cloudflare

  %% HAProxy
  %% Still exists for Pages and Registry but isn't a priority
  chef-repo ----> com-haproxy
  haproxy-cookbook ----> com-haproxy

  %% Documentation
  platforms -- CI workflow --> Documentation
```

Pros and Cons

Pros

- Consolidation of the definitions of our limits into one place. This will streamline creating or changing any rate limits.
- The catalog can be pumped into documentation and published. Changes here will be reflected in docs and streamline the documentation process, creating more transparency for users and keeping documentation up to date.
- We are not rewriting throttling functionality, and will keep our defense in depth model of rate limiting at different layers of the product.
- We can build this to enable customisation of rate limits per namespace / project / user / customer.

- This paves the way for making it easier for support to identify which limit a customer is hitting: because all limits are defined in a single place, we could let the application advertise an identifier of the rule that is causing the request to be limited.

Cons

- Not all application rate limiting configuration is currently exposed through an API. We will need to work with product / backend engineers to implement the Application level changes.

Considerations

Is the scope of this the existing GitLab.com or Cells?

Both! Anything we create needs to support making management of rate limiting simple across the existing GitLab.com and the Cells architecture, as it's likely they will run in parallel for the duration of these improvements, and we don't want to limit ourselves to only making improvements for the future.

Improvements to support the configuration of using actors such as customers, organisations, and namespaces will be essential to ensure any improvements made to rate limiting configuration will work across Cells.

Setting Cloudflare Rate Limits

GitLab.com, Cells, and Dedicated all utilise Cloudflare at the edge of our network. We can modify the configuration for GitLab.com to use the Cloudflare WAF Rules Terraform module. Since this was built to be extensible, we can leverage this to simplify our configuration, and provide a quick win and proof of concept for this concept.

Setting Application Rate Limits

Rate Limit configuration within the application will be slightly more difficult. At present these limits are configured within the Rails app, some of which have an API but not all of them. Going forward, we should enforce that all limits are configured and managed in the same way, whether that be through an API, or configuration files. Implementing the framework outlined in the Next Rate Limiting Architecture design document will help us set this direction.

Publishing Rate Limits

One advantage of having all rate limits declared in YAML in a centralized source of truth for limits that are imposed on requests to GitLab.com. This YAML can be parsed then published to our documentation or handbook, allowing for greater transparency for our users. An added bonus is that any changes to the service would automatically update our documentation, and save manual updates needing to be published.

Confidential Rate Limits

When implementing the rate limiting interface and surfacing rate limiting rules and thresholds in documentation, we need to introduce a mechanism to keep some of the configuration SAFE, in

cases such as bypasses and rules introduced to protect us from malicious actors. In these cases, the configuration can be considered confidential as some of the rules may contain customer IP addresses, or details pertaining to the nature of an attack we are protecting against.

New Services

When new services are added to GitLab.com, sometimes they are created without rate limits, and it can be hard to add limits in after the fact without impact to users. Any new services created should also be added to our Rate Limit Interface. We would add this as a step to the Production Readiness check, and set suggested defaults for rate limits for new services.

Runway

Runway enables engineers to streamline the deployment of projects outside of the main GitLab codebase. Anything that we create should take into account how we might integrate with Runway as part of ensuring these services automatically have rate limits configured when created.

Self-Managed

Any of the improvements we make to the GitLab Application will need to work with Self-Managed instances, which if we make modifications to the Application limits to support configuration files, then self-managed customers should be able to benefit from these improvements.

Dedicated

The Cells architecture is based on Dedicated tooling, and with improvements being made to utilise the Cloudflare WAF module, that lays the foundations for supporting improvements here too.

Cloud Connector

Cloud Connector rate limits are configured in Cloudflare to throttle the consumption of non-horizontally scalable resources such as AI vendor limits. As part of Phase 1 to simplify our edge network configuration, we'll need to ensure Cloud Connector rate limits are included in these improvements.

Alternative Solutions

During the proposal phase of this design document we have had a lot of excellent discussions, many of which may lead to scope creep were we to include them as part of the proposal, however are important to capture for posterity.

HAProxy versus Cloudflare

Cloudflare is consistently used for GitLab.com, Dedicated, and Cells. Therefore, focusing on moving any existing configuration out of HAProxy and into Cloudflare will immediately be an improvement on our existing configuration.

Note: Some services, such as Pages and Registry still rely on HAProxy as they are not routed

through Cloudflare.

Building a rate limiting service using Cloudflare workers

The HTTP Routing Service was considered as a potential place to house this rate limiting implementation, as it was thought that it would be application-aware. However the HTTP Router (Cloudflare worker) doesn't provide any additional intelligence over what we already have with our existing Cloudflare setup, as it will be reading the headers and path to make a routing decision, so this will not gain us any benefit over the existing rate limit architecture.

Rate Limits per Cell

An idea had been proposed around introducing rate limits on a per Cell basis, however, this is not a sustainable solution. For example, if we were to introduce a higher rate limit threshold for one cell based on a customer's need, then if that customer were moved to a different Cell at a later date, it's possible the rate limit configuration could be missed, or become outdated. Therefore tying rate limit configuration on a per Cell basis is not the direction we are proposing to go.

Instead, we should focus on rate limits on a per organization, project, or namespace level.

The HTTP Router configuration for Cells means that the existing GitLab.com Cloudflare rules will automatically apply across all Cells.

Cell Reference Architecture

The reference architecture/ size of a Cell will likely be smaller than the existing GitLab.com platform, therefore we should probably consider introducing different maximum rate limits based on this reduced capacity.

Organization IDs in URLs

A potential future improvement that is out of scope of this initial proposal would be to include organization IDs in URLs as query parameters. If we were to do this, then that would enable us to introduce organization based rate limits at the Cloudflare layer, rather than needing to implement these rules as Application rate limits.

Breaking Authentication and Authorization out of the Application

One future forward thought was to break AuthN and AuthZ out of the GitLab application and into their own service. If we were to do this, we could implement a quota or rate-limit service as a bundled feature or supplementary service. If we were to do this, we could utilise a combination of our API Gateway (whether that is HAProxy or a more modern cloud-native gateway) and utilise the authentication service to enforce the most specific rate limits at the gateway level before they reach the rest of the GitLab application.

SECTION: engineering/architecture/design-documents/rate_limiting_simplification/schema.md

<!-- Design Documents often contain forward-looking statements -->
<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

Summary

This design document proposes a standardized YAML schema to serve as a single source of truth for rate limiting configuration across GitLab. Currently, rate limits are defined inconsistently across multiple systems including Cloudflare, HAProxy, RackAttack, and Application Rate Limiters, making them difficult to manage, document, and troubleshoot. The fragmented approach has led to confusion for users and support engineers, challenges in implementing new limits, and difficulty in maintaining an accurate inventory of applied limits.

The proposed schema will consolidate all rate limiting definitions into a version-controlled configuration format that can be used consistently across GitLab.com, Dedicated environments, and self-managed instances. This work is a critical component of Phase 2 of the "Simplifying Rate Limiting Configuration" design document and supports the broader "Next Rate Limiting Architecture" blueprint by providing a concrete implementation path for standardizing how rate limits are defined and enforced.

By centralizing rate limit configuration, we aim to improve transparency for users and support teams, simplify configuration management for production engineers, enable automated documentation generation, and ensure consistent enforcement of policies across all GitLab platforms while maintaining our defense-in-depth approach of limiting at multiple layers.

Motivation

The GitLab application currently lacks a standardized way to define and maintain rate limiting configurations. This design document focuses specifically on creating a YAML schema that will serve as the foundation for consistent rate limit definitions across our systems.

Currently, there are several challenges that a standardized YAML schema would address:

- We lack a formal schema for defining rate limits, leading to inconsistent implementations across different parts of the application and infrastructure.
- Different rate limiting implementations (RackAttack, Application Rate Limiter, etc.) use different formats and approaches to configuration (file, API, ...), making it difficult to understand and manage limits holistically.
- Without a well-defined schema, it's challenging to extend our rate limiting capabilities or add new types of limits while maintaining consistency.
- There's no standardized way to validate that rate limit configurations are correct before deployment, increasing the risk of errors when making changes.
- The lack of a schema makes it difficult to automatically generate documentation, leading to discrepancies between actual configurations and what's documented.
  - This presents challenges during incidents, as can be hard to know if something is rate-limited, and how to change this rate limit if there is any.
  - This is also a pain point for customers for whom it can be very difficult to understand if they are being rate-limited, and why.

The YAML schema proposed in this document will provide a clear structure for defining rate limits that can be used across GitLab systems. It will establish conventions for critical attributes like limit thresholds, scope definitions, and enforcement actions (i.e. throttle, log, or disable).

This work directly supports the "Next Rate Limiting Architecture" blueprint's goal to "build a framework to define and enforce limits in GitLab Rails" by establishing the foundational schema upon which that framework will be built.

Goals

- Define a comprehensive YAML schema for rate limiting configuration that can support all current and future rate limit types
- Create a format that is human-readable, easily maintainable, and properly version-controlled
- Enable validation of configurations before deployment to prevent errors
- Focus on supporting Rack Attack rate limits only initially
- Design the schema to be extensible for future rate limiting enhancements (e.g. Cloudflare WAF)
- Ensure the schema supports automatic documentation generation to keep user-facing documentation accurate

Non-Goals

- Implementation of the code that consumes this schema
- Migration of existing rate limit configurations to use this schema
- Building a user interface for managing configurations defined in this schema (phase 3)
- Defining specific rate limit values (the schema defines the structure, not the actual limit values)
- Creating a centralized rate limiting service (phase 3)

Proposal

We propose to create a standardized YAML schema that defines the rate limit parameters for GitLab's various rate limiting systems. This focused schema will serve as a single source of truth for rate limit values, without attempting to reimplement or reconfigure the underlying rate limiting mechanisms.

The schema will be organized as a list of all rate limits, with a simple structure that can be easily understood, maintained, and extended for new rate limiting systems.

Project

We will create a new group gitlab-com/kinds that will serve as a global kind registry which represents all schemas, present or future, within GitLab.

Within this group, we will create a new kind project gitlab-com/kinds/rate-limits that will host:

- The YAML schema definition itself
- Example configurations

- Documentation
- Tooling for schema validation and configuration consumption (Ruby library, Jsonnet library, Terraform module...)

This project will be the canonical source of the rate limiting schema, making it easy for various GitLab components to reference a specific version of the schema.

It will be public, to allow self-managed customers to benefit from the improvements made to our rate limiting configuration, as they could then have the option to utilize the schema themselves.

The tooling for testing and releasing the schema will be based on the existing one from the tenant-model-schema project and standardized in common-ci-tasks so that it can be reused in any future kind projects.

Semantic Versioning

The schema will follow strict semantic versioning principles:

- Major Version (X.y.z): Incremented for breaking changes that require consumers to update their implementation; these should be carefully considered and avoided as much as possible, and should be rolled out in multiple stages following the Expand/Contract pattern.
- Minor Version (x.Y.z): Incremented for backward-compatible additions to the schema.
- Patch Version (x.y.Z): Incremented for backward-compatible bug fixes or documentation updates.

Each release will be properly tagged in the repository, allowing consumers to pin to specific versions or version ranges.

Publication

For each release, the schema and its documentation will be published to a static site via GitLab Pages to facilitate its consumption for schema validation.

Schema Structure Example

yaml
```
$schema: https://gitlab-com.gitlab.io/kinds/rate-limits/v1.0.0/rate-limits.yaml
ratelimits:
  gitbasicauth:   Unique identifier for this rate limit
    description: Limits basic authentication requests per IP to prevent abuse
    actors:   Actors (ipaddress, user, group)
      - ipaddress
    featurecategory: systemaccess
    enabled: true
    action: block   Action (block, log)
    limit:
      threshold: 600   Number of requests
      period: "1m"   Time period (s=seconds, m=minutes, h=hours, d=days)
```

```
  projectexports:   Unique identifier for this rate limit
    description: Limits number of project exports a user can initiate
    actors:   Actors (ipaddress, user, group)
      - user
    featurecategory: importers
    enabled: true
    action: log   Action (block, log)
    limit:
      threshold: 5   Number of requests
      period: "1d"   Time period (s=seconds, m=minutes, h=hours, d=days)
```

Rate Limits Configuration Flow

```
mermaid
flowchart TB
    subgraph Rate Limit Kind Project
        S[Rate Limiting Configuration YAML Schema]
        RL[Ruby Library]
        TM[Terraform Module]
    end

    subgraph Cloudflare
        WAFCells[Cells WAF]
        WAFSaaS[SaaS WAF]
    end

    subgraph GitLab SaaS
        subgraph Rails Application
            RL -. Include .-> CRLSaaS[class RateLimits]
            CRLSaaS -- Configure --> RASaaS[Rack Attack]
            CRLSaaS -- Configure --> ARLSaaS[Application Rate Limits]
        end

        subgraph Rate Limits Configuration Project
            S -. Validate .-> CfgSaaS[Rate Limits YAML Configuration]
        end

        CfgSaaS -- Fetch --> HelmSaaS["Helm (k8s-workloads/gitlab-com)"] -- Configure -->
CRLSaaS
        CfgSaaS -- Fetch --> TFSaaS["Terraform (config-mgmt)"] -- Configure --> WAFSaaS
        TM -. Include .-> TFSaaS
    end

    subgraph Cell / Dedicated Tenant
        subgraph Rails Application
            RL -. Include .-> CRLCells[class RateLimits]
            CRLCells -- Configure --> RACells[Rack Attack]
            CRLCells -- Configure --> ARLCells[Application Rate Limits]
        end
```

```
    subgraph Instrumentor
      S -. Validate .-> CfgCells[Rate Limits YAML Configuration]
      TMCells["Tenant Model"] -- Generate --> CfgCells
      CfgCells --> HelmCells[Helm] -- Configure --> CRLCells
      CfgCells --> TFCells[Terraform] -- Configure --> WAFCells
      TM -. Include .-> TFCells
    end
  end
```

SECTION: engineering/architecture/design-documents/rearchitect_approval_rules/_index.md

{{< engineering/design-document-header >}}

Summary

This blueprint outlines a major re-architecture of GitLab's approval rules system, which aims to enhance performance and flexibility.

The proposed changes seek to address 3 key objectives:

- Boost the efficiency of approval rule modifications, particularly in projects with numerous open Merge Requests (MRs).
- Streamline updating approval rules across various levels, including groups, projects, and MRs.
- Lay the groundwork for the future implementation of group-level approval rules.

Motivation

By implementing a more efficient architecture, it is expected that GitLab will improve the performance of approval rule updates, enabling and reflecting those changes across various levels of entities. This rearchitecting also lays the groundwork for future implementations, such as group-level approval rules, ensuring GitLab can evolve to meet the needs of complex organizational structures. Users will benefit from faster approval rule updates, improved rule management across different organizational levels, enhanced overall performance, and a more seamless workflow.

Current limitations

Changes to project-level rules are not reflected in any MR-level rules after they have been created. To mitigate this problem we allow project administrators to restrict the editing of approval rules at the MR-level. This means project-level rules are applied directly to MRs, and any updates to them are not propagated. See prevent editing approval rules in merge requests for more details.

Goals

Ideally, we would like to be able to:

- Allow the addition of group approval rules. Without this new architecture, propagating new and modified group approval rules to sub groups, projects and MRs would be very expensive. With this new architecture, that operation becomes more manageable.
- Allow users to modify inherited rules within MRs, projects, and sub-groups.
- Facilitate updating of open MR level rules from the project level where overriding is allowed but has not occurred. For more details, see https://gitlab.com/gitlab-org/gitlab/-/issues/254958.
- Retain inherited approval rules at the MR level, which have been modified.
- Retain existing behavior including:
  - TODO: list all current behavior that we wish to retain.

Non-Goals

TBD

Proposal

The proposal aims to improve the performance, flexibility, and scalability of GitLab's approval rules system by introducing a new architecture. The main points of the proposal are:

- Create new models and tables to support the system needs.
- Change the relationship structure: instead of copying rules attributes, create associations between merge requests and approval rules.
- Maintain existing behavior: when "Prevent editing approval rules in merge requests" is enabled, continue using project-level rules directly.
- Handle rule overrides: when a rule is overridden at the MR-level, create a new ApprovalRule and associate it with the merge request.
- Retain compliance information: keep the existing ApprovalMergeRequestRule table to store the state of approval rules when a merge request is merged. We may rename the table to better describe the purpose.

A new data model structure, proposed first by @patrickbajao in this comment:

- MergeRequests::ApprovalRule.
- MergeRequests::ApprovalRulesGroup - join table associating MergeRequests::ApprovalRule to a Group.
- MergeRequests::ApprovalRulesProject - join table associating MergeRequests::ApprovalRule to a Project.
- MergeRequests::ApprovalRulesMergeRequest - join table associating MergeRequests::ApprovalRule to a MergeRequest

Existing Data Models

mermaid
erDiagram
  Project ||--o{ MergeRequest: " "
  Project ||--o{ ApprovalProjectRule: " "
  ApprovalProjectRule }o--o{ User: " "
  ApprovalProjectRule }o--o{ Group: " "
  ApprovalProjectRule }o--o{ ProtectedBranch: " "

```
  MergeRequest ||--|| ApprovalState: " "
  ApprovalState ||--o{ ApprovalWrappedRule: " "
  MergeRequest ||--o{ Approval: " "
  MergeRequest ||--o{ ApprovalMergeRequestRule: " "
  ApprovalMergeRequestRule }o--o{ User: " "
  ApprovalMergeRequestRule }o--o{ Group: " "
  ApprovalMergeRequestRule ||--o| ApprovalProjectRule: " "
```

Proposed Data Models

```
mermaid
erDiagram
  "MergeRequests::ApprovalRule" ||--|| "MergeRequests::ApprovalRulesGroup": " "
  "MergeRequests::ApprovalRule" }o--o{ User: " "
  "MergeRequests::ApprovalRule" }o--o{ UserGroup: " "
  "MergeRequests::ApprovalRule" }o--o{ ProtectedBranch: " "
  "MergeRequests::ApprovalRule" ||--o{ "MergeRequests::ApprovalRulesProject": " "
  "MergeRequests::ApprovalRule" ||--o{ "MergeRequests::ApprovalRulesMergeRequest": " "
  "MergeRequests::ApprovalRulesGroup" }o--|| Group: " "
  "MergeRequests::ApprovalRulesProject" }o--|| Project: " "
  "MergeRequests::ApprovalRulesMergeRequest" }o--|| MergeRequest: " "
  MergeRequest }o--|| Project: " "
  Project }o--|| Group: " "
  MergeRequest ||--|| ApprovalState: " "
  MergeRequest ||--o{ Approval: " "
  ApprovalState ||--o{ ApprovalWrappedRule: " "
```

Design and implementation details

The development plan is defined in the Rearchitecting Approval Rules Epic.

Alternative Solutions

TBD

SECTION: engineering/architecture/design-documents/repository_backups/_index.md

{{< engineering/design-document-header >}}

<!-- For long pages, consider creating a table of contents. The [TOC]
function is not supported on docs.gitlab.com. -->

Summary

This proposal seeks to provide an out-of-a-box repository backup solution to
GitLab that gives more opportunities to apply Gitaly specific optimisations. It

will do this by moving repository backups out of backup.rake into a coordination worker that enumerates repositories and makes per-repository decisions to trigger repository backups that are streamed directly from Gitaly to object-storage.

The advantages of this approach are:

- The backups are only transferred once, from the Gitaly hosting the physical repository to object-storage.
- Smarter decisions can be made by leveraging specific repository access patterns.
- Distributes backup and restore load.
- Since the entire process is run within Gitaly existing monitoring can be used.
- Provides architecture for future WAL archiving and other optimisations.

This should relieve the major pain points of the existing two strategies:

- backup.rake - Repository backups are streamed from outside of Gitaly using RPCs and stored in a single large tar file. Due to the amount of data transferred these backups are limited to small installations.
- Snapshots - Cloud providers allow taking physical storage snapshots. These are not an out-of-a-box solution as they are specific to the cloud provider.

Motivation

Goals

- Improve time to create and restore repository backups.
- Improve monitoring of repository backups.

Non-Goals

- Improving filesystem based snapshots.

Filesystem based Snapshots

Snapshots rely on cloud platforms to be able to take physical snapshots of the disks that Gitaly and Praefect use to store data. While never officially recommended, this strategy tends to be used once creating or restoring backups using backup.rake takes too long.

Gitaly and Git use lock files and fsync in order to prevent repository corruption from concurrent processes and partial writes from a crash. This generally means that if a file is written, then it will be valid. However, because Git repositories are composed of many files and many write operations may be taking place, it would be impossible to schedule a snapshot while no file operations are ongoing. This means the consistency of a snapshot cannot be guaranteed and restoring from a snapshot backup may require manual intervention.

WAL may improve crash
resistance and so improve automatic recovery from snapshots, but each repository will likely still require a majority of voting replicas in sync.

Since each node in a Gitaly Cluster is not homogeneous, depending on replication factor, in order to create a complete snapshot backup all nodes would need to have snapshots taken. This means that snapshot backups have a lot of repository data duplication.

Snapshots are heavily dependent on the cloud provider and so they would not provide an out-of-a-box experience.

Downtime

An ideal repository backup solution would allow both backup and restore operations to be done online. Specifically we would not want to shutdown or pause writes to ensure that each node/repository is consistent.

Consistency

Consistency in repository backups means:

- That the Git repositories are valid after restore. There are no partially applied operations.
- That all repositories in a cluster are healthy after restore, or are made healthy automatically.

Backups without consistency may result in data-loss or require manual intervention on restore.

Both types of consistency are difficult to achieve using snapshots as this requires that snapshots of the filesystems on multiple hosts are taken synchronously and without repositories on any of those hosts currently being mutated.

Distribute Work

We want to distribute the backup/restore work such that it isn't bottlenecked on the machine running backup.rake, a single Gitaly node, or a single network connection.

On backup, backup.rake aggregates all repository backups onto its local filesystem. This means that all repository data needs to be streamed from Gitaly (possibly via Praefect) to where the Rake task is being run. If this is CNG then it also requires a large volume on Kubernetes. The resulting backup tar file then gets transferred to object storage. A similar process happens on restore, the entire tar file needs to be downloaded and extracted on the local filesystem, even for a partial restore when restoring a subset of repositories. Effectively all repository data gets transferred, in full, multiple times between multiple hosts.

If each Gitaly could directly upload backups it would mean only transferring repository data a single time, reducing the number of hosts and so the amount of data transferred over all.

Gitaly Controlled

Gitaly is looking to become self-contained and so should own its backups.

backup.rake currently determines which repositories to backup and where those backups are stored. This restricts the kind of optimisations that Gitaly could apply and adds development/testing complexity.

Monitoring

backup.rake is run in a variety of different environments. Historically backups from Gitaly's perspective are a series of disconnected RPC calls. This has resulted in backups having almost zero monitoring. Ideally the process would run within Gitaly such that the process could be monitored using existing metrics and log scraping.

Automatic Backups

When backup.rake is set up on cron it can be difficult to tell if it has been running successfully, if it is still running, how long it took, and how much space it has taken. It is difficult to ensure that cron always has access to the previous backup to allow for incremental backups or to determine if updating the backup is required at all.

Having a coordination process running continuously will allow moving from a single-shot backup strategy to one where each repository determines its own backup schedule based on usage patterns and priority. This way each repository should be able to have a reasonably up-to-date backup without adding excess load to any Gitaly node.

Updated Repositories Only

backup.rake packages all repository backups into a tar file and generally has no access to the previous backup. This makes it difficult to determine if the repository has changed since last backup.

Having access to previous backups on object-storage would mean that Gitaly could more easily determine if a backup needs to be taken at all. This allows us to waste less time backing up repositories that are no longer being modified.

Point-in-time Restores

There should be a mechanism by which a set of repositories can be restored to a specific point in time. The identifier (backup ID) used should be able to be determined by an admin and apply to all repositories.

WAL (write ahead log)

We want to be able to provide infrastructure to allow continuous archiving of the WAL. This means providing a central place to stream the archives to and being able to match any full backup to a place in the log such that repositories can be restored from the full backup, and the WAL applied up to a specific point in time.

WORM

Any Gitaly accessible storage should be WORM (write once, read many) in order to prevent existing backups being modified in the case an attacker gains access to a nodes object-storage credentials.

The pointer layout
currently used by repository backups relies on being able to overwrite the pointer files, and as such would not be suitable for use on a WORM file store.

WORM is likely object-storage provider specific:

- AWS object lock
- Google Cloud WORM retention policy.
- MinIO object lock

bundle-uri

Having direct access backup data may open the door for clone/fetch transfer optimisations using bundle-uri. This allows us to point Git clients directly to a bundle file instead of transferring packs from the repository itself. The bulk repository transfer can then be faster and is offloaded to a plain http server, rather than the Gitaly servers.

Proposal

The proposal is broken down into an initial MVP and per-repository coordinator.

MVP

The goal of the MVP is to validate that moving backup processing server-side will improve the worst case, total-loss, scenario. That is, reduce the total time to create and restore a full backup.

The MVP will introduce backup and restore repository RPCs. There will be no coordination worker. The RPCs will stream a backup directly from the called Gitaly node to object storage. These RPCs will be called from backup.rake via the gitaly-backup tool. backup.rake will no longer package repository backups into the backup archive.

This work is already underway, tracked by the Server-side Backups MVP epic.

Per-Repository Coordinator

Instead of taking a backup of all repositories at once via backup.rake, a backup coordination worker will be created. This worker will periodically enumerate all repositories to decide if a backup needs to be taken. These decisions could be determined by usage patterns or priority of the repository.

When restoring, since each repository will have a different backup state, a timestamp will be provided by the user. This timestamp will be used to determine which backup to restore for each repository. Once WAL archiving is implemented, the WAL could then be replayed up to the given timestamp.

This wider effort is tracked in the Server-side Backups epic.

Design and implementation details

MVP

There will be a pair of RPCs BackupRepository and RestoreRepository. These RPCs will synchronously create/restore backups directly onto object storage. backup.rake will continue to use gitaly-backup with a new --server-side flag. Each Gitaly will need a backup configuration to specify the object-storage service to use.

Initially the structure of the backups in object-storage will be the same as the existing pointer layout.

For MVP the backup ID must match an exact backup ID on object-storage.

The configuration of object-storage will be controlled by a new config config.backup.gocloudurl. The Go Cloud Development Kit tries to use a provider specific way to configure authentication. This can be inferred from the VM or from environment variables.
See Supported Storage Services.

Alternative Solutions

<!--
It might be a good idea to include a list of alternative solutions or paths considered, although it is not required. Include pros and cons for each alternative solution/path.

"Do nothing" and its pros and cons could be included in the list too.
-->

SECTION: engineering/architecture/design-documents/reverse-grpc-tunnel-workspaces-and-ci/_index.md

<!-- Design Documents often contain forward-looking statements -->

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

Summary

Configuring and deploying the Workspace proxy with a valid SSL certificate and
domain name record is a very complicated process and is a requirement for using
Workspaces today.

This design document explains how we can re-use our
GitLab Agent (KAS)
architecture to avoid all of this setup and tunnel in via KAS.

This document also describes how this could be used to get a GitLab VS Code fork connected to a running CI Job as an additional benefit.

While this document is related to
https://gitlab.com/gitlab-com/content-sites/handbook/-/mergerequests/10811

Proposal

In short, we propose that we tunnel HTTP and SSH traffic over gRPC (using KAS)
to access our deployed workspaces. This would be an alternative option to our
workspace proxy and it has the following benefits over the workspace proxy:

1. It doesn't require the user to deploy any ingress/certmanager/proxy to K8s
1. It doesn't require the user to register a domain name
1. It doesn't require the user to deal with SSL certificates

Initially we would provide this as an optional alternative to the workspace
proxy but still maintain support for the workspace proxy. This would unblock
other ways of deploying workspaces (see below).
Longer term we can determine whether or not we want to maintain both options
depending on user feedback.

In addition we found that it was easy to extend this tunnel to be useful for
debugging CI jobs using the GitLab VS Code fork so that is also included in this proposal.
This idea of tunneling may provide an alternative network transport
to support
Interactive Web Terminals
which currently relies on direct network access to the Runner Manager and is likely a considerable barrier for adoption.

Technical details

The main idea of this proposal is to make use of the
agentk -> KAS
gRPC connection to tunnel HTTP requests to a GitLab VS Code fork running on the same

container as the agent. Since agentk was built with a different purpose in mind (communicating with the K8s API) we are likely to build a different agent binary while trying to re-use as much of the server-side components as possible.

The HTTP tunnel has already been demonstrated in https://gitlab.com/gitlab-org/cluster-integration/gitlab-agent/-/mergerequests/2084 and was a simple extension of existing HTTP tunnel behaviour in KAS. We still need to build a new SSH tunnel in KAS to allow for SSH connectivity to the Workspace, but we do not anticipate there should be any technical limits preventing us from doing this. The largest architectural part of this work would be building an SSH server into KAS which authenticates users based on our preferred SSH authentication mechanisms. This work is not mutually exclusive with
ongoing work to change the way our SSH proxy works
as this network tunnel could be used for websocket-based SSH tunneling.

This idea was demonstrated in this
video demo which is composed of
POC changes in the following merge requests:

1. https://gitlab.com/gitlab-org/gitlab/-/mergerequests/176478
1. https://gitlab.com/gitlab-org/cluster-integration/gitlab-agent/-/mergerequests/2084
1. https://gitlab.com/gitlab-org/workspaces/gitlab-workspaces-tools/-/mergerequests/19

This idea was demonstrated in this
video demo which is composed of
POC changes in the following merge requests:

1. https://gitlab.com/gitlab-org/cluster-integration/gitlab-agent/-/mergerequests/2084
1. https://gitlab.com/gitlab-org/workspaces/gitlab-workspaces-tools/-/mergerequests/19

Agent lifecycle

In this proposal we will be introducing a new type of agent. We'll call it a "tunneling agent" for now but we may come up with a different name once we start developing it.

These tunneling agents will be short lived ephemeral agents that only exist for the lifecycle of a single workload that needs to be tunnelled into. In the case of Workspaces we will create these agents when we create a workspace and we will expire/destroy them after the workspace is terminated. They will have a short lived agent token that is injected into the workspace via environment variables.

These agents will not be shown in other parts of the GitLab application and do not require agent config files like normal agentk agents.

Background

This proposal is based on experimental proof of concept work done as part of https://gitlab.com/gitlab-org/gitlab/-/issues/505764 to explore ways to minimise the amount of effort to get started with Workspaces.

During the investigation we found that
KAS already
has most of the building blocks for this network tunnel and as such would be the most efficient option for getting this to production. KAS was originally built as a way to tunnel into customer's K8s clusters, and this is reflected in the name, but the same techniques can easily be applied to tunneling into any customer workloads so it seems like a natural extension of this service.

Related work for deploying workspaces to VMs

The work complements
another proposal
for how we might also run workspaces without Kubernetes at all, but this proposal focuses solely on the network tunneling behaviour that will be used for both of these. It will be possible to ship either of these without the other and provide incremental user value, and as such we created separate proposals.

In practice this proposal for network tunneling may be easiest to implement first as a smaller iteration to unblock deploying workspaces via CI.

Iteration plan

1. New KAS tunnel
   1. GitLab tunnel agents model
   2. New endpoint in KAS server
   3. New agent binary
1. Workspace integration
   1. Workspace creation optionally creates the tunnel agent
   1. Workspace creation optionally installs the tunnel agent
   1. Workspace init optionally starts the tunnel agent (using injected env var for short lived token)

Alternatives considered

Build a whole new service instead of KAS

Since KAS was not built specifically for this purpose, it is tempting to build a new service with this single responsibility. We may still decide to do that if we find that we just can't make that fit. But it is not our first choice as there are many technical complexities in building a service that meets these requirements. Here are some of the not-so-obvious tricky details that KAS has already solved:

1. Clustering: Many to many relationships between agents and servers means that

the user's HTTP request may not reach the same server which has a connection to the agent they are trying to reach. KAS solves this by clustering (via Redis) to locate the correct server with the client connection and forwarding the request to that server.
2. Connection pooling: Since the gRPC streams can only be opened by the client (in this case the agent) it is not possible to just open a new stream (or send a new gRPC message) for every HTTP request which comes into the server. KAS solves this by keeping open a buffer of idle gRPC streams to the agent and it opens additional streams when all streams are in use for HTTP requests.
3. Authorization: KAS already integrates with GitLab to authenticate and authorize access to specific resources and agents.

Deploying load balancers/Ingress for the customer

Since the original motivation for this work was to find alternatives to Kubernetes which are simpler for customers to set up it was also considered that we might build tooling to provision the load balancers and Ingress for the customer. This would simplify the setup but it does come with tradeoffs. It still requires the customer to register a domain name and there are still complexities with generating SSL certificates. One of the biggest challenges with using Let's Encrypt to generate SSL certificates is that there are limits on the number of certificates you can generate within a period of time. Using our KAS ingress avoids any such limits as we can use a single wildcard certificate for all workspaces. Each workspace would have unique domain, which is some subdomain of worskspacerootdomain.example.com (for example).

SECTION: engineering/architecture/design-documents/runner_admission_controller/_index.md

{{< engineering/design-document-header >}}

The GitLab admission controller (inspired by the Kubernetes admission controller concept) is a proposed technical solution to intercept jobs before they're persisted or added to the build queue for execution.

An admission controller can be registered to the GitLab instance and receive a payload containing jobs to be created. Admission controllers can be mutating, validating, or both.

- When mutating, mutable job information can be modified and sent back to the GitLab instance. Jobs can be modified to conform to organizational policy, security requirements, or have, for example, their tag list modified so that they're routed to specific runners.
- When validating, a job can be denied execution.

Motivation

To comply with the segregation of duties, organizational policy, or security requirements, customers in financial services, the US federal government market segment, or other highly regulated industries must ensure that only authorized users can use runners associated with

particular CI job environments.

In this context, using the term environments is not equivalent to the definition of the environment used in the GitLab CI environments and deployments documentation. Using the definition from the SLSA guide, an environment is the "machine, container, VM, or similar in which the job runs."

An additional requirement comes from the Remote Computing Enablement (RCE) group at Lawrence Livermore National Laboratory. In this example, users must have a user ID on the target Runner CI build environment for the CI job to run. To simplify administration across the entire user base, RCE needs to be able to associate